

예외, 제네릭, 컬렉션, 함수형

오늘의 강의 학습 요약

본 강의에서는 자바 예외 처리의 계층 구조와 처리 방식(try-catch-finally, throws), 명시적 예외(throw) 및 사용자 정의 예외의 필요성을 정리하고, 런타임 예외와 비런타임(체크) 예외를 왜 구분하는지 컴파일 시점/실행 시점 관점에서 설명했습니다. 특히 런타임 예외는 try-catch로 "처리"하기보다 조건 체크로 "예방"하는 것이 핵심이라는 점을 강조했습니다.

이어서 Object 기반 범용 Box 예제를 통해 제네릭이 필요한 이유(컴파일 타임 타입 체크, 형변환 제거)를 문제-해결 흐름으로 다루고, 컬렉션 API(Set/List/Map)의 계층과 대표 구현(HashSet/ArrayList/HashMap), 핵심 메소드, 출력/순회 방식(향상된 for, Iterator 등)을 실습 중심으로 정리했습니다. 마지막으로 스트림 API의 목적(중간 처리/최종 처리)과 그 기반이 되는 표준 함수형 인터페이스(java.util.function)의 분류(Consumer/Supplier/Function/Operator/Predicate) 및 람다 구현 패턴을 예제로 확인했습니다.

1. 예외 처리와 런타임, 체크 예외

예외는 Exception 계열을 중심으로 계층 구조를 가지며, 처리 방식은 try-catch-finally 또는 throws 위임으로 구성되고 런타임 예외는 조건 체크로 예방하는 관점이 핵심입니다.

자바 예외는 계층 구조로 설계되어 다형성이 적용됩니다. 최상위에는 Throwable 이 있으나, 일반적인 애플리케이션에서 “처리 가능한 예외”의 중심은 Exception 계열로 봅니다. Error 계열은 복구 대상으로 다루지 않는다는 점에서 예외 처리 범위에서 보통 제외됩니다.

예외 처리 방식은 크게 두 갈래입니다. 예외 발생 지점에서 try-catch-finally 로 직접 처리하거나, 메소드 선언부의 throws 로 호출자에게 처리를 위임합니다. catch 와 throws 모두 예외 클래스가 필요하며, 여러 예외를 다루면 다중 catch 또는 throws 에 복수 예외를 선언하는 형태가 됩니다.

예외 클래스가 계층을 가지므로 상위 타입으로 하위 예외를 포괄 처리할 수 있습니다. 예를 들어 Exception 은 하위 예외를 모두 받을 수 있으나, 여러 catch 를 사용할 때는 하위 예외부터 배치해야 합니다. 상위 예외를 먼저 잡으면 아래 catch 는 도달 불가가 되어 컴파일 에러가 발생합니다.

finally 는 예외 발생 여부와 무관하게 수행되어야 하는 정리 작업에 사용됩니다. try-finally 형태도 가능하며, 이는 “예외 처리” 목적이거나 “반드시 실행되어야 하는 코드 보장” 목적이 강합니다.

명시적 예외 발생은 throw new 예외클래스(...) 형태로 수행됩니다. 예를 들어 잔액 부족과 같이 시스템 장애는 아니지만 업무 규칙상 예외로 다루고 싶은 상황은 개발자가 예외를 “발생”시키는 방식으로 모델링할 수 있습니다. 이때 도메인에 특화된 상황은 API가 모두 제공할 수 없으므로 사용자 정의 예외 클래스를 직접 만들어 사용하는 흐름이 필요합니다.

상속 관계에서 오버라이딩할 때 throws 는 “축소” 방향 제약이 있습니다. 부모가 던진 예외보다 같거나 더 하위(더 구체적) 예외만 선언할 수 있고, 경우에 따라 아예 던지지 않도록 줄이는 것도 가능합니다. 또한 런타임 예외를 던지는 선언과 체크 예외 선언은 계층 및 규칙에 따라 제한됩니다.

런타임 예외(RuntimeException 계열)와 체크 예외(비런타임 예외)는 “발생 시점과 강제성”이 다릅니다. 런타임 예외는 실행 중 발생하며, 컴파일 시점에 예외 처리를 강제하지 않습니다. 반면 체크 예외는 컴파일 시점에 처리(try-catch 또는 throws)를 강제하며, 처리하지 않으면 컴파일 에러가 납니다.

런타임 예외는 ArithmeticException , ArrayIndexOutOfBoundsException , NullPointerException , ClassCastException 등이 대표적이며, 핵심 대응은 예외 처리 구문이 아니라 조건 체크로 사전 예방하는 것입니다. 예를 들어 0 나누기 전 검사, 인덱스 범위 검사, null 검사 등이 이에 해당합니다.

체크 예외는 IOException , SQLException 등이 대표적이며, 파일 I/O나 DB 연동처럼 외부 요인으로 실패가 발생할 수 있는 코드는 컴파일 단계에서 예외 처리가 강제됩니다. 따라서 이러한 영역은 try-catch 또는 throws 설계가 필수입니다.

구분	대표 예외	발생 시점	처리 강제	권장 대응
런타임 예외	<code>NullPointerException</code> , <code>ArithmeticException</code> 등	실행 시점	강제 아님	조건 체크로 예방합니다.
체크 예외	<code>IOException</code> , <code>SQLException</code> 등	컴파일 시점에 강제 요구	강제 됨	try-catch 또는 throws로 설계합니다.

핵심 키워드

`#Exception`

`#try-catch-finally`

`#throws`

`#RuntimeException`

`#SQLException`

2. 제네릭스: Object 한계와 타입 안정성

Object 기반 범용 저장은 컴파일 타임 타입 검증이 불가능하고 형변환이 강제되며, 제네릭은 컴파일 시점 치환을 통해 타입 제한과 형변환 제거를 제공합니다.

임의 타입 데이터를 저장하는 `Box` 를 `Object` 필드로 구현하면 문자열, 정수 등 어떤 값이든 담을 수 있습니다. 그러나 요구사항이 “문자열만 저장”처럼 구체화되면 문제가 발생합니다.

첫째, 잘못된 타입이 저장되어도 컴파일 단계에서 검출되지 않습니다. 예를 들어 문자열만 저장해야 하는데 실수로 정수를 넣어도 컴파일은 통과하며, 오류는 실행 시점에 드러납니다.

둘째, 값을 꺼낼 때 원래 타입의 기능을 쓰려면 반드시 형변환이 필요합니다. `Object` 에는 `String.length()` 같은 문자열 전용 멤버가 없으므로 `(String)` 캐스팅이 필요하고, 이 과정에서 잘못된 타입이면 `ClassCastException` 이 발생합니다.

제네릭스는 이러한 문제를 컴파일 시점 타입 체크로 이동시켜, 개발 시점에 오류를 발견하게 합니다. 또한 타입이 확정되므로 반환값을 원래 타입으로 형변환할 필요가 없어집니다.

제네릭 표기는 꺾쇠(`<>`)로 하며, `T` (타입), `E` (요소), `R` (리턴) 같은 타입 파라미터를 사용합니다. `K`, `V` 는 `Map`에서 Key/Value를 나타냅니다. 타입 파라미터는 “레퍼런스 타입만” 받을 수 있으므로 기본형 `int` 는 불가하고 `Integer` 같은 래퍼 타입을 사용해야 합니다.

제네릭 타입은 컴파일 타임에 지정 타입으로 “치환”되는 관점이 중요합니다. 예를 들어 `Box<String>` 이라면 내부 `T` 가 컴파일 시점에 `String` 으로 치환되어, `setValue(10)` 같은 코드는 컴파일 에러가 발생합니다.

제네릭 적용 위치는 클래스, 인터페이스, 메소드 레벨이 대표적입니다. 또한 제네릭 타입 파라미터는 정적 컨텍스트에서의 제약 등 사용 규칙이 있으므로, 이후 컬렉션 API에서 활용 방식으로 자연스럽게 익히는 흐름이 적절합니다.

```
class Box2<T> {  
    private T value;  
    public void setValue(T value) { this.value = value; }  
    public T getValue() { return value; }  
}
```

// 사용 예: Box2<String>은 String만 허용됩니다.

```
Box2<String> b = new Box2<>();
```

```
b.setValue("홍길동");
```

```
// b.setValue(10); // 컴파일 에러
```

```
String s = b.getValue(); // 형변환 불필요
```

핵심 키워드

#제네릭스

#타입 파라미터

#컴파일 타임 치환

#형변환

#래퍼 클래스

3. 컬렉션 API: Set, List, Map 구조와 사용

컬렉션은 크기 변경과 풍부한 메소드를 제공하며, Set은 무순서, 중복불가, List는 순서, 중복허용, Map은 키-값 쌍과 빠른 키 기반 조회가 핵심입니다.

자바의 데이터 저장 방식은 변수(1개), 배열(여러 개, 고정 크기), 컬렉션(여러 개, 크기 변경 가능)으로 정리됩니다. 배열은 크기 변경이 불가능하고 자체 유틸 메소드가 부족해 Arrays 유틸리티에 의존하는 측면이 있습니다.

컬렉션은 java.util 중심으로 제공되며, 인터페이스 기반 계층 구조를 가집니다. Collection 인터페이스를 기반으로 Set, List가 파생되며, Map은 Collection 계열과 별도의 계통입니다. 실사용은 인터페이스가 아니라 대표 구현체를 생성해 사용합니다.

분류	인터페이스	대표 구현	저장 방식 핵심
Set 계열	Set<E>	HashSet<E>	순서가 없고 중복 저장이 되지 않습니다.
List 계열	List<E>	ArrayList<E>	순서가 있고 중복 저장이 가능합니다.
Map 계열	Map<K,V>	HashMap<K,V>	키-값 쌍으로 저장하며 키로 빠르게 조회합니다.

Set은 무순서 특성 때문에 “몇 번째 요소” 개념이 없어 개별 조회 메소드가 제한적입니다. 출력/순회는 toString(), 향상된 for, Iterator 기반 반복이 대표적입니다. 또한 중복이 자동 제거되므로 중복 제거 용도로 유용합니다.

List는 순서가 있어 인덱스 기반 조회(get)와 삽입(add(index, value)), 수정(set), 삭제(remove)가 가능합니다. 또한 subList(from, to) 로 부분 리스트를 얻을 수 있으며, to 인덱스는 포함되지 않는 규칙이 중요합니다.

Map은 put 으로 저장하고 get(key) 로 조회합니다. 키가 없으면 null 이 반환되며, 동일 키로 put 하면 값이 덮어써지면서 “수정” 효과가 발생합니다. 키 전체는 keySet() 으로 Set 형태로 얻고, 값 전체는 values() 로 Collection 형태로 얻어 반복할 수 있습니다.

컬렉션에는 제네릭을 적용해 저장 타입을 제한하는 사용이 일반적입니다. 예를 들어 HashSet<String> 은 문자열만 저장되며, 잘못된 타입은 컴파일 단계에서 차단됩니다.

List의 remove 는 오버로드로 인해 주의가 필요합니다. List<Integer> 에서 remove(1) 은 “값 1 삭제”가 아니라 “인덱스 1 삭제”로 해석될 수 있으므로, 값 삭제는 remove(Integer.valueOf(1)) 또는 new Integer(1) 처럼 객체로 전달해야 합니다.

컬렉션 객체 생성 방식도 다양합니다. `Arrays.asList(...)` 는 가변 인자(varargs)를 받아 빠르게 리스트를 만들 수 있지만, 반환 리스트는 “고정 크기(fixed-size)”여서 `add / remove` 가 `UnsupportedOperationException` 을 발생시킵니다. `Map.of(...)` 도 빠른 초기화가 가능하나 불변(수정 불가) 맵을 만들므로 `put` 이 예외를 유발합니다.

```
List<String> a = Arrays.asList("A", "B", "C"); // 고정 크기
// a.add("D"); // UnsupportedOperationException

Map<String, Integer> m = Map.of("K1", 10, "K2", 20); // 불변 맵
// m.put("K3", 30); // UnsupportedOperationException
```

핵심 키워드

`#HashSet`

`#ArrayList`

`#HashMap`

`#Iterator`

`#Arrays.asList`

4. 스트림 개요와 표준 함수형 인터페이스

스트림은 컬렉션 데이터를 흐름으로 변환해 중간 처리(필터, 정렬, 중복제거)와 최종 처리(합계, 평균, 최대, 최소)를 수행하며, 처리는 `java.util.function`의 함수형 인터페이스와 람다로 기술됩니다.

컬렉션 API는 저장/삭제/조회/크기 같은 기본 기능에 초점이 있고, 합계, 평균, 최대/최소 같은 통계 처리나 필터링, 정렬 같은 데이터 처리 기능은 직접 구현이 필요했습니다. 이러한 처리 요구를 지원하기 위해 JDK는 스트림 API를 제공하며, 컬렉션의 데이터를 “흘러보내는 통로”로 모델링합니다.

스트림 처리 흐름은 중간 처리와 최종 처리로 구분됩니다. 중간 처리에서는 정렬, 중복 제거, 조건 필터링 등을 수행하며, 최종 처리에서는 합계, 평균, 최댓값, 최솟값 등 결과를 산출합니다. 이때 각 처리 단계는 “기능”이므로 메소드 형태로 표현되어야 하고, 그 메소드 시그니처는 결국 파라미터/리턴 조합의 제한된 형태를 가집니다.

표준 함수형 인터페이스는 `java.util.function` 패키지에 있으며, 스트림이 요구하는 다양한 “함수(동작)”를 미리 정의해 둔 것입니다. 주요 분류는 다음과 같이 정리됩니다.

분류	의미	핵심 추상 메소드 형태	특징
Consumer	파라미터 O, 리턴 X	<code>accept(...)</code>	소비(처리)만 하고 결과를 반환하지 않습니다.
Supplier	파라미터 X, 리턴 O	<code>get()</code>	값을 공급(생성)해 반환합니다.
Function	파라미터 O, 리턴 O	<code>apply(...)</code>	입력을 다른 값으로 변환해 반환합니다.
Predicate	파라미터 O, 리턴 boolean	<code>test(...)</code>	조건 판정에 사용되며 필터링에 활용됩니다.
Operator	Function의 특수형	<code>apply(...)</code>	입력/출력 타입이 동일한 변환을 나타냅니다.

Consumer는 `Consumer<T>` (파라미터 1개)와 `BiConsumer<T,U>` (파라미터 2개)처럼 파라미터 개수에 따라 분화됩니다. 또한 `IntConsumer`, `DoubleConsumer` 처럼 기본형에 특화된 인터페이스도 제공합니다.

Supplier는 `Supplier<T>` 처럼 리턴 타입이 제네릭인 형태가 대표이며, `BooleanSupplier`, `DoubleSupplier`, `LongSupplier` 처럼 기본형 리턴에 특화된 타입도 존재합니다.

Function은 `Function<T,R>` 처럼 입력 타입과 출력 타입을 분리해 표현합니다. `BiFunction<T,U,R>` 는 입력이 2개인 버전이며, `DoubleToIntFunction` 처럼 기본형 변환 전용도 존재합니다.

Operator는 `UnaryOperator<T>`, `BinaryOperator<T>` 처럼 “입력과 출력 타입이 같다”는 제약을 부여한 Function 계열의 특수 형태입니다. 예를 들어 `BinaryOperator<T>` 는 두 입력 T를 받아 T를 반환합니다.

Predicate는 `Predicate<T>` 가 대표이며, 리턴 타입이 반드시 `boolean` 으로 고정됩니다. `BiPredicate<T,U>`, `IntPredicate`, `LongPredicate` 등도 같은 논리로 확장됩니다.

가변 인자(varargs) 문법 ... 도 함께 정리되었습니다. `Arrays.asList(T... a)` 나

`System.out.printf(String format, Object... args)` 처럼 인자 개수가 가변인 메소드 시그니처에서 사용되며, 내부적으로 배열로 취급됩니다.

함수형 인터페이스의 구현은 익명 클래스로도 가능하지만, 실전(특히 스트림)에서는 람다 표현식이 핵심입니다. 강의에서는 `Consumer`, `BiConsumer`, `ObjDoubleConsumer`, `Supplier`, `DoubleSupplier`, `Function` 등을 익명 클래스 → 람다 순으로 구현하며 문법 축약 규칙(타입 생략, 괄호 생략, 단문 중괄호/return 생략)을 반복 적용했습니다.

```
// Consumer<String>
Consumer<String> c = s -> System.out.println(s);
c.accept("hello");

// Supplier<String>
Supplier<String> sup = () -> "홍길동";
String name = sup.get();

// Function<String, Integer>
Function<String, Integer> f = s -> s.length();
int len = f.apply("hello");
```

핵심 키워드

`#Stream API`

`#java.util.function`

`#Consumer`

`#Supplier`

`#Predicate`